

Dokumentation und kurze Anleitung für die Abgabe:

Ausgewählte Themen der Webprogrammierung



Wichtig!!!:

Link zum Abgabeordner: <https://cloud.hs-harz.de/s/7aitbtqLyGjqLrc>

Name: Leonard Mottke

Matrikelnummer: m30904

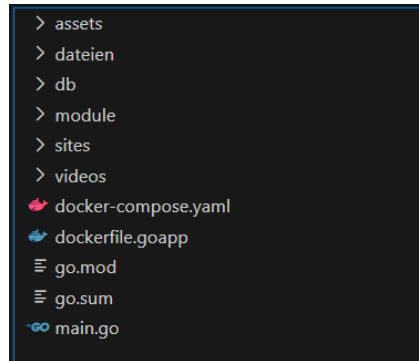
Inhaltsverzeichnis

1.	Vorwort	3
2.	Planung der Abgabe	3
3.	Code Beispiele.....	3
3.1.	Template	3
3.2.	Go-code	4
4.	Herausforderungen	5
5.	Design.....	5
6.	Verbesserungen für die Zukunft und Ziele.....	6
7.	Anleitung zum Starten der Anwendung	7
8.	Fazit.....	7
9.	Eigenständigkeitserklärung und KI-einsatz.....	7
9.1.	KI-einsatz	7
9.2.	Eigenständigkeitserklärung	8

1. Vorwort

Dies ist die Dokumentation zur Abgabe „Ausgewählte Themen der Webprogrammierung“. In diesem Dokument gebe ich Einblicke, wie ich beim Erstellen meiner Go-Webanwendung vorgegangen bin und wie ich zu meinen Ergebnissen gekommen bin.

2. Planung der Abgabe



Die Aufgabe bestand darin, mein Portfolio in einer Go-basierten Anwendung umzusetzen. Da ich bereits eine Portfolio-Webseite in HTML im Rahmen der Vorlesung „Portfolio / Showreel“ erstellt hatte, nutzte ich diese als Grundlage und erweiterte sie. Zuerst habe ich mir angeschaut, wie ich die Ordnerstruktur am besten anlege. Am Ende waren es sechs Hauptordner; die meisten enthalten darüber hinaus mehrere Unterordner.

Ich habe auch eingeplant, dass es statische Elemente gibt, da ich Videos habe, die recht groß sind. So ist der Video-Ordner statisch, die Assets sind statisch, und im Sites-Ordner gibt es einen weiteren Ordner, der „statisch“ heißt, in dem alle restlichen statischen Elemente vorhanden sind.

3. Code Beispiele

3.1. Template

```
1  {{define "content"}}
2  <section class="portfolio">
3    {{range .}}
4    <div class="tile">
5      <div class="pic">
6        <!-- Bild anzeigen -->
7        
8      </div>
9      <div class="text">
10       {{.Text}}
11     </div>
12   </div>
13   {{end}}
14 </section>
15 {{end}}
16
```

Hier habe ich die Struktur der Content-Blöcke definiert. Die ID kommt aus der Datenbank, damit das zugehörige Bild geladen werden kann. Zusätzlich wird aus der Struktur in model.go zugegriffen, damit die Inhalte eingefügt werden.

```

sites > templ > <> sliderseite.gohtml > ...
1  {{define "sliderseite"}}
2  <!DOCTYPE html>
3  <html lang="en">
4      {{template "head" .}}
5      <body>
6          <!-- Wrapper für den gesamten Inhalt -->
7          <div class="wrapper">
8              <!-- Header-Bereich -->
9              {{template "nav" .Nav}}
10             <!-- Hauptinhalt -->
11             <main>
12                 {{template "slider" .Slider}}
13                 <br />
14                 <h1>{{.Titel}}</h1>
15                 <!-- Portfolio-Bereich mit mehreren "tiles" -->
16                 {{template "content" .ContentBloেকে}}
17             </main>
18             <!-- Ende des Portfolio-Bereichs -->
19
20             <!-- Footer-Bereich mit Impressum und Datenschutz -->
21             {{template "footer"}}
22         </div>
23         <!-- Ende des Wrappers -->
24
25         <!-- Lädt das JavaScript am Ende, um die Ladezeiten zu verbessern -->
26         <script src="../../assets/js/script.js"></script>
27         <script src="../../assets/js/slider.js"></script>
28     </body>
29 </html>
30 {{ end }}
31

```

In diesen Templet habe ich die Slider Seiten struktur erstellt. Alles was man zu einen Seperaten Templet machen konnte wird hier hineingefügt. Es gibt zwei Slider Seiten einmal index(Home) und Projekte, es gibt zwei Video seiten und eine TabLinkSeite. Alle sind ähnlich aufgebaut.

3.2. Go-code

```

// Der Server startet angeschlossen auf Port 8080
func main() {
    err := db.VerbindenDB()
    if err != nil {
        log.Fatalf("Probleme mit verbinden der Datenbank! %s \n", err.Error())
    }
    defer func () {
        if err := db.VerbindungTrennenMongoDB(); err != nil {
            log.Fatalf("Probleme beim Trennen der MongoDB!")
        }
    }()

    router := gin.Default()

    router.LoadHTMLGlob(filepath.Join(templateOrder, templateDatei))
    router.Static("/sites/statisch", "../../sites/statisch")
    router.Static("/assets", "../../assets")
    router.Static("/videos", "../../videos")
    router.GET("/", seitenHandler)
    router.GET("/seite/:name", seitenHandler)
    router.GET("/image/:id", bildbereitstellen)
    router.GET("/pdf/:name", pdfbereitstellen)
    log.Print("Gestartet bei den Port: 8080")
    err = router.Run(":8080")
    if err != nil {
        log.Fatal(err)
    }
}

```

In der main.go wird in der Funktion main die Verbindung zur Mongo-Datenbank hergestellt und initialisiert. Die Router werden mit der Gin-Bibliothek aufgesetzt und verweisen an den entsprechenden Adressen auf die zuständigen Go-Funktionen.

4. Herausforderungen

```
// fillDatenbank leert zunächst die Seiten-Collection und befüllt sie neu.  
// Dazu wird eine ZIP-Datei geöffnet, Markdown-Dateien aus dem definierten  
// Ordner extrahiert, YAML-Daten geparsed und je nach Seitentyp  
// (VideoSeite, SliderSeite, TabLinkboxSeite) verarbeitet.  
// Bilder und PDFs werden zusätzlich in GridFS gespeichert.
```

```
func fillDatenbank() error{  
    seitenSammlung := mongodb.Collection(seitencollection)  
    seitenSammlung.Drop(context.Background())  
    log.Println("Datenbank wird befüllt!")  
}
```

```
// BildStruktur repräsentiert ein Bildobjekt.  
// Die ID wird nach dem Upload in GridFS gesetzt.  
// Der Dateiname wird aus der YAML-Datei übernommen.  
type BildStruktur struct {  
    ID        string `bson:"id,omitempty" json:"id,omitempty"`  
    Dateiname string `bson:"dateiname" json:"dateiname" yaml:"dateiname"`  
}
```

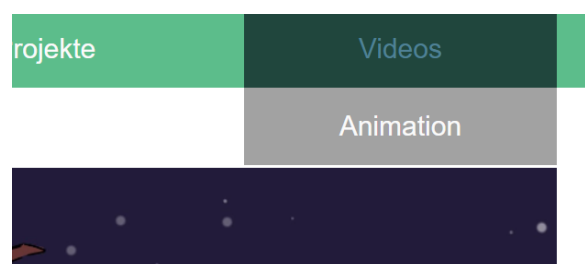
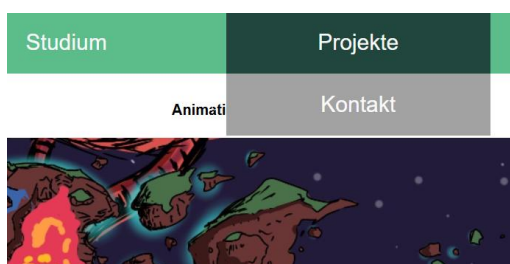
Es gab mehrere schwierige Stellen, zum Beispiel die Funktion fillDatenbank in der mongo.go. An manchen Stellen hatte ich statt eines Strings fälschlicherweise primitive.ObjectID verwendet, was die ID-Struktur veränderte. Weitere Probleme waren Rechtschreibfehler, die ich nicht sofort gefunden habe. Die Markdown-Dateien mit YAML-Inhalten hatte ich zuerst nicht korrekt eingebunden. Außerdem gab es einfache Tippfehler und nicht korrekt geschriebene Dateinamen.

5. Designe

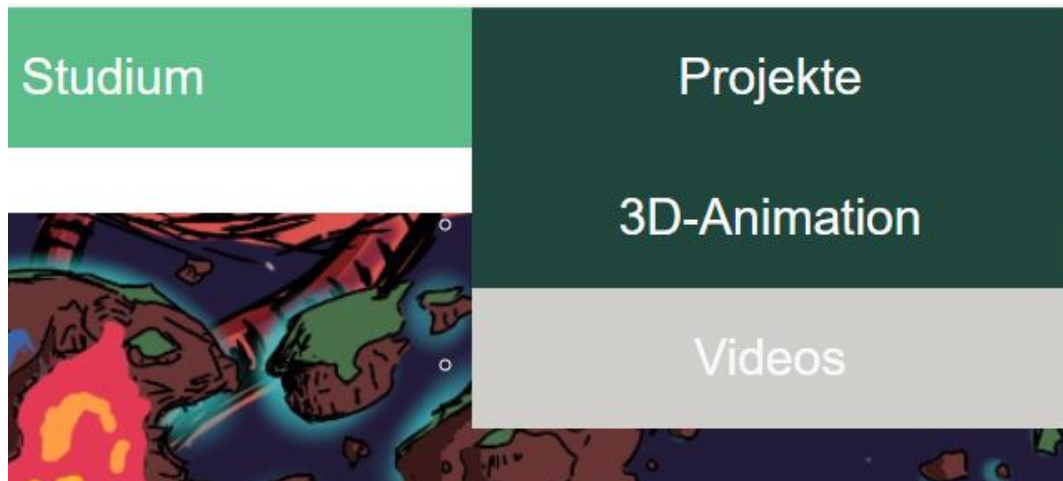
Ich habe das Design meiner vorherigen Webseite (Portfolio / Showreel) an einigen Stellen verändert. Zuvor hatte ich zwei Dropdown-Bereiche, jetzt nur noch einen. Auch die Navigation hat sich leicht verändert.

Dropdown:

Vorher:



Nachher:



Navigation:

Vorher:

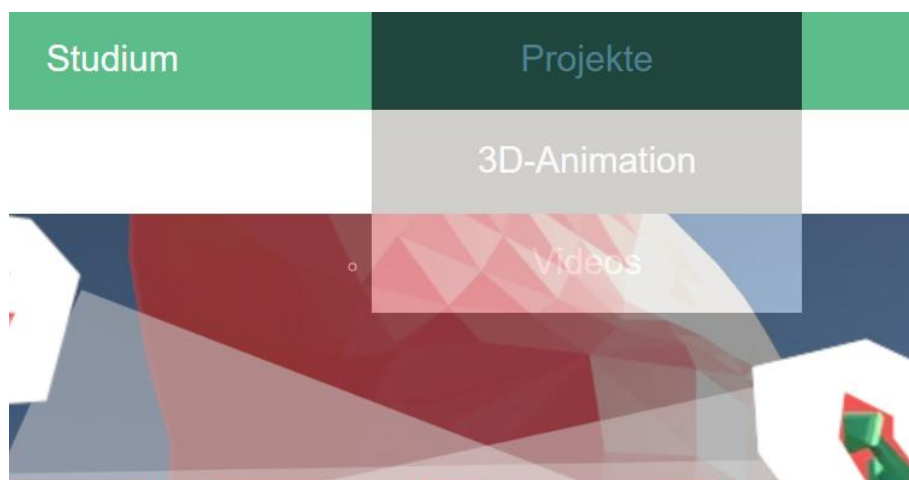


Nachher:



6. Verbesserungen für die Zukunft und Ziele

Ich habe zwei Dinge, die ich an meinem Go-Portfolio in Zukunft verbessern möchte. Zum einen ist es mein Ziel, die Anwendung im Internet zu betreiben, sodass sie dauerhaft verfügbar ist. Voraussetzung dafür ist, dass ich mit meinem Portfolio zufrieden bin. Zum anderen möchte ich das Dropdown-Menü in der Navigation verbessern, da es an manchen Stellen durch den Slider überdeckt wird.



7. Anleitung zum Starten der Anwendung

Schritt für Schritt Anleitung:

1. Stelle sicher, dass Docker installiert und gestartet ist (z. B. Docker Desktop).
2. In VS Code oder einen Andern Code editor: Datei → Ordner öffnen... und wähle Portfolio_Abgabe_Go.
3. Menü Terminal → New Terminal.
4. Im Terminal sicherstellen, dass du im Projektordner Portfolio_Abgabe_Go bist.
5. Kurz prüfen, ob Docker läuft `docker-compose --version` (Optional)
6. Anwendung bauen und starten `docker-compose up --build`
7. Anwendung im Browser öffnen `http://localhost:8080`
8. Beenden: im Terminal mit `Ctrl + C` stoppen.
9. herunterfahren: `docker-compose down`

8. Fazit

Es gab viele Probleme bei der Erstellung meiner Anwendung, und ich habe oft meine Grenzen ausreizen müssen. Meine Gesamtzeit für diese Abgabe beträgt fast drei Wochen, und ich bin trotzdem recht stolz auf mein Endprodukt, das ich abgeben kann, auch wenn es noch ein paar Macken hat und noch nicht ganz der Feinschliff da ist.

Was ich aus dieser Abgabe mitnehme, ist, dass mir das Planen der Strukturen geholfen hat und es wahrscheinlich nicht so glatt gelaufen wäre, wenn ich dies anders gemacht oder das Projekt vielleicht sogar nicht geschafft hätte.

9. Eigenständigkeitserklärung und KI-einsatz

9.1. KI-einsatz

Ich habe ChatGPT für verschiedene Sachen benutzt. Zum einen habe ich es zum Kommentieren des Codes genutzt und mir auch manche Sachen erklären lassen. Auch meine Rechtschreibung habe ich von der KI überprüfen lassen.

Meistens habe ich es aber beim Finden von Fehlern, die aufgetreten sind, benutzt.

Ich wusste beim Seitenhandler zum Beispiel nicht, wie man eine spezifische Seite ohne Parameter zurückgibt, was ich durch ChatGPT dann herausgefunden habe.

Ein weiteres Problem war das Laden der PDFs aus der Datenbank. Da wusste ich nicht genau, wie die Unterschiede zum Laden der Bilder sind, weshalb ich auch hier ChatGPT genutzt habe. Im Endeffekt haben sich die Funktionen nicht wirklich unterschieden.

9.2. Eigenständigkeitserklärung

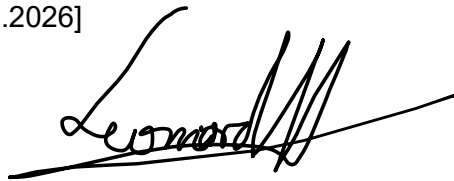
Eigenständigkeitserklärung

Hiermit erkläre ich, dass ich die vorliegende Arbeit selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe.

Alle sinngemäß und wörtlich übernommenen Textstellen aus fremden Quellen wurden kenntlich gemacht.

[Wernigerode], den [19.02.2026]

[Leonard Mottke]

A handwritten signature in black ink, appearing to read 'Leonard Mottke', with a long horizontal line extending to the right.